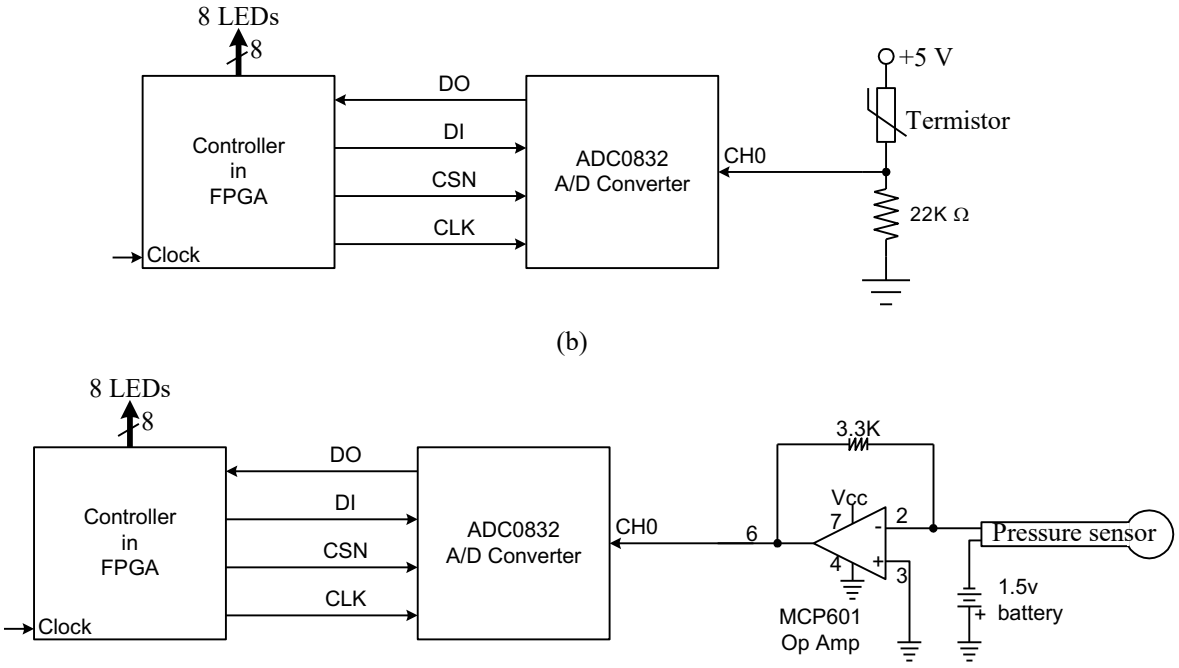


图 9.41 ADC 芯片与模拟传感器相连接：(a) 用于测试的电位计；(b) 热敏电阻式温度传感器；(c) 一个采用 MCP601 运算放大器的压力传感器。

9.9 实时时钟的 I²C 总线控制器

在这一小节中我们会设计和实现用来控制实时时钟芯片的微型控制器。实时时钟芯片与使用标准 I²C 总线协议的微型控制器相连接。微型控制器作为上层连接到 I²C 总线。实时控制芯片又作为下层连接到 I²C 总线。使用 I²C 总线作为通信渠道，主体的微型控制器可以发送和接收来自从体的数据。

9.9.1 工作原理

I²C 总线作为一个简单的双向连续总线，可以支持多个主体与从体。它由两条线组成：一个序列双向数据通道（SDA）和一个序列双向时钟通道（SCL）。在 I²C 总线定义当中，定义了 100kHz 最大时钟速率的标准模式和 400kHz 最大速率的快速模式。

每一个连接到 I²C 总线项链的硬件都是可以通过一个唯一的地址来软件寻址到的。在硬件之间的主从关系一直存在着。那些通过控制总线接入来控制数据的发送和接受的叫做主体。发送数据到总线的被叫做发送器，从总线接收数据的叫做接收器。

同一个 I²C 总线可以接入多于一个的主体和多于一个的从体。但是总线每次只能被一个主体控制，而且它负责产生序列时钟（SCL）和通过启动和停止消息的传递控制总线接入。

我们的系统由唯一一个主体构成（FPGA 上的微型控制器）和一个从体（Maxim DS3232 实时时钟芯片）如图 9.42 所示。连接主体和从体的 SDA 和 SCL 的线都是开路，都要用 5.6K 欧的上拉电阻接到 VCC。不管多少硬件是连接到这根总线上的，每根总线都只需要一个上拉电阻就够了。注意到主体控制器的实现不一定满足 I²C 总线协议的所有描述。所以如果你把这个用到其他的 I²C 总线连接方式上，主体的控制器也许会产生错误状态。

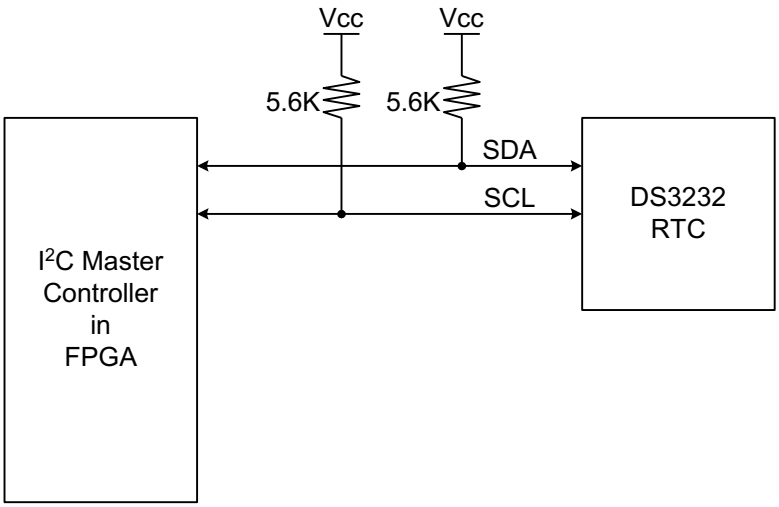


图 9.42 I²C 总线系统带有一个 FPGA 上的 I²C 主体控制器和一个从体的实时时钟芯片

当 SCL 和 SDA 线都是 1 的时候，I²C 总线是空闲状态。主体进入 START 状态（SDA 线从高变低同时 SCL 保持高电平如图 9.43(a)），启动数据传输。进入 START 状态后，总线被认为是在忙碌状态的。在 START 状态之后，主体发送一个从体地址给总线。这个地址是 7 个 bit，后面加上 1 个 bit 用来表示数据的方向（R/W）。其中 0 代表主体向从体写，1 代表主体向从体读。控制 SCL 线的主体，会在 SDA 线上发送这些 bits，每 1 个 SCL 线的时钟周期发送 1 个 bit。SDA 的线的值只能在 SCL 线为低电平的时候被改变。

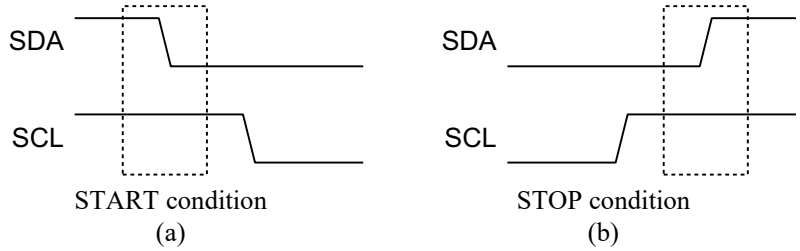


图 9.43 START(a)和 STOP(b)条件同时被主体启动。当 SDA 从高变低而 SCL 维持高电平时，START 条件发生。当 SDA 从低到高而 SCL 维持高电平时，STOP 条件发生。只有两种条件中 SDA 线可以改变电平，而且要是 SCL 为高电平时候才可以。

被寻址的那个从体硬件则在 SDA 线上发送一个确认 bit。具体方法是第 9 个 SCL 线的时钟周期中将 SDA 线拉低如图 9.44 所示。方向 bit（R / W）决定了是主体还是从体是下一个序列数据的发送者。

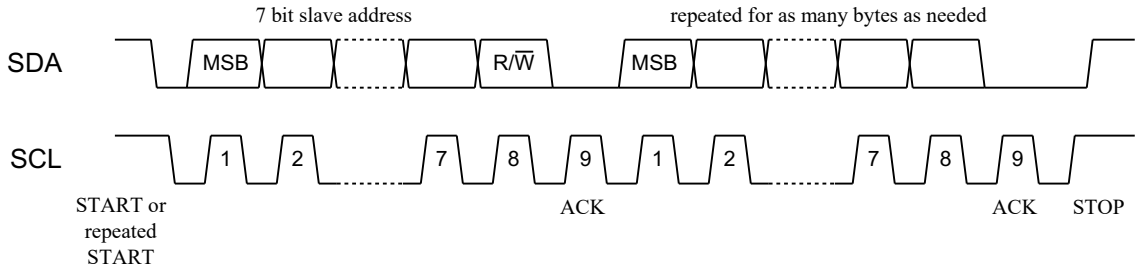


图 9.44 主体通过发送 START 条件启动数据传输。对于每 8 个 bit 的传输数据，接收器在第 9 个时钟通过拉低 SDA 线发送一个确认信号。唯一的例外是当主体接收器想要终止数据传输的时候，主体（同时也是接收者）不会发送确认信号。数据停止时通过主体发送 STOP 条件来实现的。

每个 SDA 线上发送的字节都是 8bit 长的。除了 START 和 STOP 条件，当 SCL 线是高电平的时候，SDA 线不能改变。能够传输的字节数没有被限制。每个字节必须带一个确认 bit。主体来产生确认 bit 的时钟脉冲。发送器在确认时钟脉冲内拉低 SDA 线来对接收到的字节产生确认。唯一的例外是当主体-接收器参与在传输中。这种情况下，主体-接收器必须告诉从体-发送者数据传输的结束，通过不产生确认信号。

为了告知数据传输的结束，主体需要当 SCL 线为高电平的时候，通过拉高 SDA 线来发送一个 STOP 条件，如图 9.43(b)所示。或者不发送 STOP 条件，取而代之的是主体重复发送 START 条件-，这样他可以在不用放弃接入总线的情况下改变数据传输的方向。

图 9.45(a)显示了主体向从体写 1 个字节的数据的情况（i.e.主体是发送者，从体是接收者）。主体启动数据传输，发送一个 START 条件，紧跟着 7bit 的从体地址外加一个 write(0)bit。在收到来自从体的确认后，主体发送一个寄存器序号让从体知道接下来的数据应该写入哪个寄存器。然后从体回复一个确认。主体然后发送数据字节到从体。在从体确认数据接受完后，主体发送一个 STOP 条件。

图 9.45(b)显示了主体从从体读取一个字节的数据（i.e. 主体是接收器，从体是发送器）。主体启动数据传输，发送一个 START 条件，紧跟着 7bit 的从体地址外加一个 write (0)bit。尽管主体想要接收一个字节，它首先需要发送寄存器的地址让从体知道主体可以从哪个寄存器中读取数据。在接收到来自从体的确认后，主体发送一个寄存器序号让从体知道从哪个寄存器去读取。从体然后再次发送确认信号。这一次主体需要发送一个重复的 START 条件因为它需要改变数据方向从写到读。这个重复的 START 信号紧跟着一个从体的地址，但这次后面跟着 read (1) 信号。这个从体确认然后发送确认字节给主体。这一次，因为主体是接收器，主体必须发送确认信号。如果主体想要接受更多的字节，它发送一个 0 来确认它。如果主体不想接受更多字节，它就不发任何东西，然后拉高 SDA。最后，主体发送 STOP 条件。

总结主体发送和主体接收的两种情况，在两种情况下，主体线发送一个 7bit 的从体地址和一个 write bit，紧接着是寄存器序列。在主体-发送器 情况(a)下，因为数据方向依旧是『写』，所以主体可以立刻发送一个数据字节。在主体-接收器情况(b)下，主体必须发送重复的 START 信号，然后反复发送从体地址和『写』bit，目的是改变数据传送的方向从写到读。然后，主体可以从从体的给定寄存器中接收数据字节。

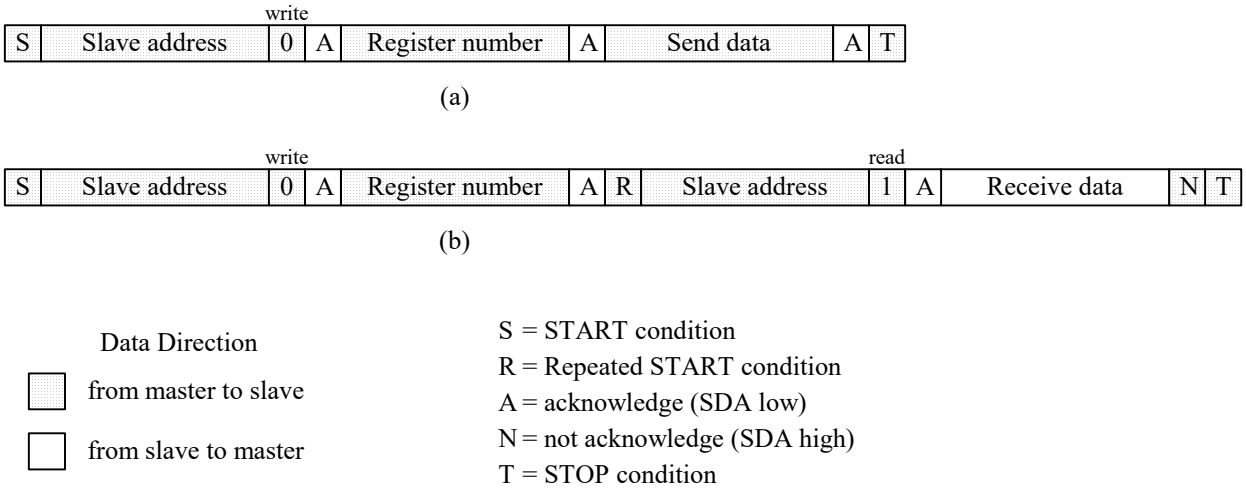


图 9.45 (a)主体发送一个字节的数据给从体 (b) 主体接收一个字节的数据来自从体 。

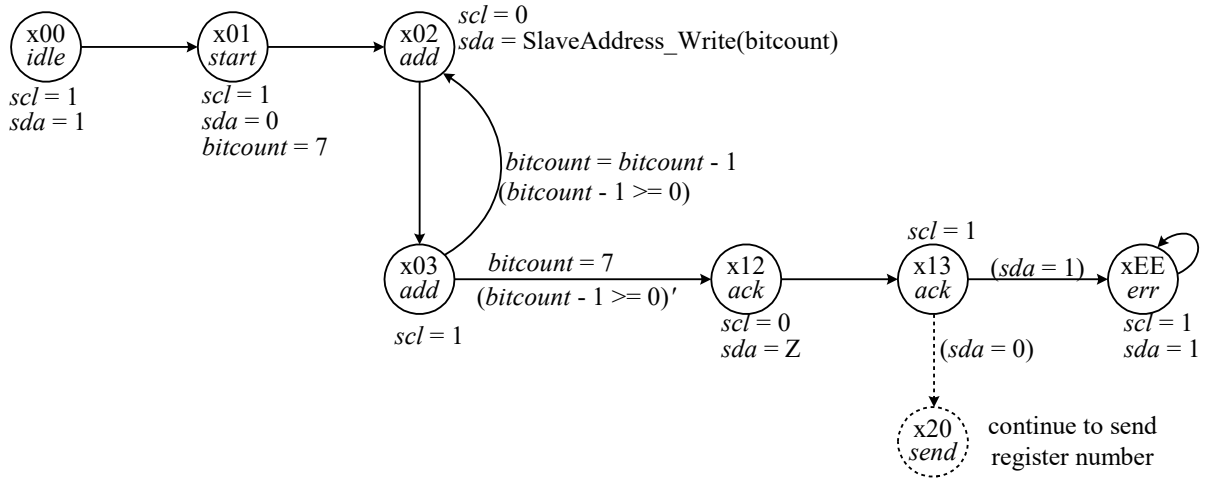
9.9.2 控制器设计

根据上述的 I²C 协议规范，我们可以得出 I²C 主体控制器的状态图。这个状态图会需要很多状态，但是其中的很多都是相同的，而且符合同样地模式。图 9.46(a) 显示了状态图最初的部分。从状态 x00 开始，FSM 转移到状态 x01 发送 Start 条件，紧跟着 7bit 的从体地址和一个 write bit。从体地址和 write bit 在之前存到了 8bit 的 SlaveAddress_Write 阵列中，然后在状态 x02 和 x03 的状态时一次一个 bit 地被移到 sda 线上。这里需要两个状态是因为 scl 线必须在 0 和 1 两个状态之间切换。在发送这 8 个 bit 之后，FSM 设置 sda 线为高

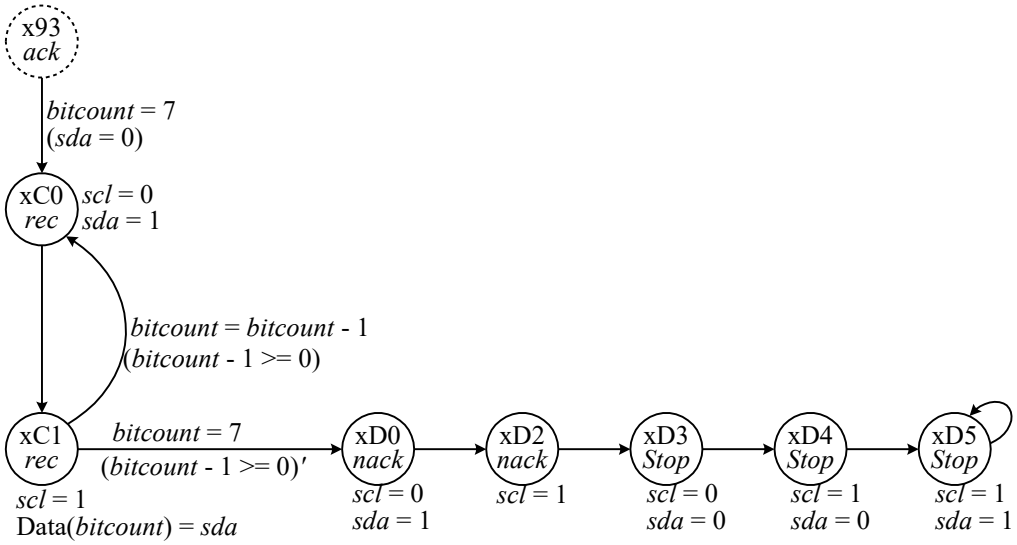
组态 Z 在状态 x12 时，来接收处于 x13 状态下的接收器发来的确认信号。如果确认信号无效(1) 那么 FSM 转移到错误状态 xEE 并且终止在那里。否则，FSM 继续转移到状态 x20 去发送寄存器序列给从体。

发送寄存器序列给从体的状态对于发送从体地址的状态 x02 和 x03 是相同的。唯一的不同在于，置入不同的 bit 值到 SDA 线当中；在这种情况下是寄存器序列而不是从体地址。

图 9.46(b)显示了从从体接收字节然后发送 Stop 条件的部分状态图。在状态 x93 中从从体接收完确认信号之后，FSM 通过设置 *bitcount* 为 7 来准备接收 8bit 的数据。FSM 在 xC0 和 xC1 之间循环 8 次之后接收了 8bit 的数据，每次讲 *sda* 上的数据接收到数据阵列当中。在接收完数据后，FSM 在 xD0 和 xD2 状态中发送一个不确认的信号后面紧跟着最后三个状态的 Stop 条件。



(a)



(b)

图 9.46 I²C 的主体控制器的部分状态图 (a)发送开始条件和从体地址 (b) 接收从体发来的数据和发送停止条件

我们利用 FSMD 模型写 HDL 代码来表示 I²C 的主体控制器。图 9.47(a)和(b)的 VHDL 代码分别对应了中的对应图 9.46 的两个状态图。整个源代码，Verilog 和 VHDL 的格式可以从书的官网上下载到。

```
output: PROCESS(CLK_200k_Hz, Go)
```

```
BEGIN
  IF(Go = '0') THEN
    -- when idle, both SDA and SCL = 1
    SCL <= '1';
    SDA01 <= '1';
    Ready <= '0';
    state <= x"00";
  ELSIF(CLK_200k_Hz'EVENT and CLK_200k_Hz = '1') THEN
    CASE state IS
    WHEN x"00" => -- Idle
      -- when idle, both SDA and SCL = 1
      SCL <= '1';   -- SCL = 1
      SDA01 <= '1'; -- SDA = 1
      Ready <= '0';
      state <= x"01";
    -- send start condition and slave address
    WHEN x"01" => -- Start
      SCL <= '1';   -- SCL stays at 1 while
      SDA01 <= '0'; -- SDA changes from 1 to 0
      Ready <= '0';
      bitcount <= 7;   -- starting bit count
      state <= x"02";
    -- send 7-bit slave address followed by R/W' bit, MSB first
    WHEN x"02" =>
      SCL <= '0';
      SDA01 <= SlaveAddress_Write(bitcount);
      state <= x"03";
    WHEN x"03" =>
      SCL <= '1';
      IF (bitcount - 1) >= 0 THEN
        bitcount <= bitcount - 1;
        state <= x"02";
      ELSE
        bitcount <= 7;
        state <= x"12";
      END IF;
    -- get acknowledgment' from slave
    WHEN x"12" =>
      SCL <= '0';
      SDA01 <= '1';
      state <= x"13";
    WHEN x"13" =>
      SCL <= '1';
      IF SDA = '1' THEN
        state <= x"EE";   -- acknowledge error
      ELSE
        state <= x"20";   -- send register address
      END IF;

    -- send 8-bit register address to slave
    WHEN x"20" =>
```

(a)

```
    -- get acknowledge signal from slave
    WHEN x"93" =>
```

```
      SCL <= '1';
      IF SDA = '1' THEN
        state <= x"EE";   -- acknowledge error
      ELSE
        bitcount <= 7;
        state <= x"C0";   -- go to receive data byte
      END IF;
    -- receive byte from RTC slave
    WHEN x"C0" =>
      SCL <= '0';
      SDA01 <= '1';
      state <= x"C1";
    WHEN x"C1" =>
      SCL <= '1';
      Data(bitcount) <= SDA; -- MSB of data read in
      IF (bitcount - 1) >= 0 THEN
        bitcount <= bitcount - 1;
        state <= x"C0";
      ELSE
        bitcount <= 7;
        state <= x"D0";
      END IF;
    WHEN x"D0" =>
      SCL <= '0';
      SDA01 <= '1'; -- send a not acknowledge' (1) signal
      state <= x"D2";
      DataOut <= Data;
    -- send a not acknowledge' (1) signal
    WHEN x"D2" =>
      SCL <= '1';
      state <= x"D3";
    -- send stop condition
    -- SDA goes from 0 to 1 while SCL is 1
    WHEN x"D3" =>
      SCL <= '0';
      SDA01 <= '0'; -- SDA starts at 0 to prepare for the 0 to 1 transition
      state <= x"D4";
    WHEN x"D4" =>
      SCL <= '1';   -- SCL goes to 1
      SDA01 <= '0'; -- SDA starts from 0
      state <= x"D5";
    WHEN x"D5" =>
      SCL <= '1';   -- SCL stays at 1 while
      SDA01 <= '1'; -- SDA goes to 1
      Ready <= '1';
      state <= x"D5";
```

图 9.47 图 9.46 中的状态图对应的两个代码: (a) 发送开始条件和从体地址 (b) 接收从体字节和发送停止信号

对 sda 信号线的赋值上需要一些特殊的注意。你可能注意到在代码中，对于所有的赋值，它用了信号名为 SDA01。SDA 信号线在 I^2C 总线中是双向的，所以它能够同时输入或者输出。其次，在 I^2C 总线中 SDA 线是开路的，所以需要一个 5.6k 欧的上拉电阻。所以为了输出逻辑 1，你需要将它设成高阻抗。为了得到高阻抗，你需要利用一个三台输出，然后给它赋值为 Z。从 SDA01 信号到 SDA 信号的真正转换发生在下述语句当中。

这句话赋值给 SDA 信号线了高阻态 Z，当内信号 SDA01 是逻辑 1 的时候。否则，SDA 信号线为逻辑 0。所以为了给 SDA 信号赋值 0 或者 1 的时候，代码赋值 0 或 1 给内部信号 SDA01，然后 SDA 信号就被置 0 或者 Z 了。

I²C 总线协议确定了标准模式下的最大的时钟速率是 100kMz。所以，我们需要确保 *scl* 信号达到这个速率。*scl* 信号由 FSM 通过不停地在两个状态下来回产生：第一个状态设置 *scl* 为 0，第二个状态设置 *scl* 为 1. 所以，对于每一个 *scl* 周期，FSM 必须经过这两个状态。为了得到 100kHz 的时钟速率，FSM 时钟速率必须快过或者等于 200kMz。如果主时钟速率是 50MHz，那么我们需要一个时钟分频器，计算 50MHz 的周期循环次数，从 0 到 250（i.e.,50,000,000 / 200,000）。为了得到 50%的占空比的 200kHz 时钟，时钟信号 CLK_200k_Hz 每 125 次触发一次(i.e., 250/2)。

9.9.3 应用实现

既然我们有了 I²C 主体控制器能够与实时时钟从体通信，我们需要一个用户界面，能够允许我们来设置实时时钟芯片的时间并将它显示在 LED 上。界面端口的顶层模块如图 9.48 所示。

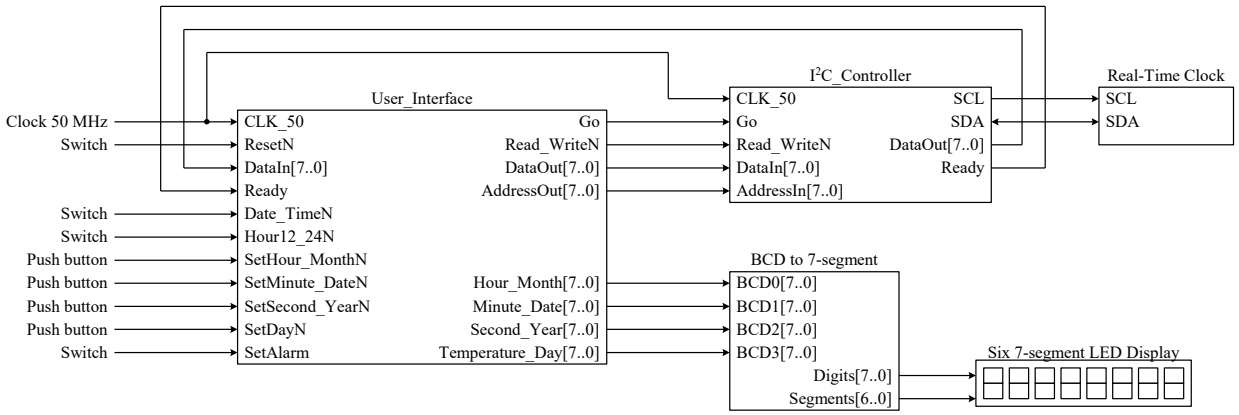


图 9.48 实时时钟的顶层模块

用户端口 FSM 有两个功能：1) 不停地向 I²C 控制器发送指令来从试试时钟芯片中读取日期和时间信号。时间和日期信息然后可以在六个 LED 上显示；2) 它等待用户开关或者按钮的输入信号。当一个开关或者按钮被处罚，FSM 通过四个控制信号线发送合适的指令给 I²C 控制器：*Go, Read_WriteN, DataIn[7..0]*和 *Address[7..0]*，来设置实时时钟芯片中相应的日期和时间寄存器。四个输出信号，*Hour_Month*，*Minute_Day, Second_Year* 和 *Temperature_Day*，会输出合适的值给 BCD 到 7 段解码器来解出正确的日期和时间信号。

用户输入包括 *Data_TimeN* 开关来选择设置日期还是时间，*Hour12_24N* 选择时间是 12 小时制还是 24 小时制，*SetHour_MonthN, SetMinute_DateN, SetSecond_YearN, SetDayN* 和 *SetAlarm* 按钮和开关来设置日期和时间信息。

9.10 USB 的 SPI 总线控制器

通用串行总线（USB）成为不同的计算机和设备彼此连接的常用方式，并且在许多情况下，取代了 RS-232 串行连接。该 USB 使得外围设备连接到计算机变得更容易和更方便用户的使用，但为了保证连接的稳健性，该协议规范做得比为 RS-232 要复杂得多。正因为如此，用 USB 连接工作相对更加复杂与困难。然而，有许多带有内部串行接口引擎（SIE）的特殊的 IC，来处理底层 USB 协议细节。

有一种这样特殊的 IC 叫做 MAX3421E USB 外设/主机控制器。它包含了所有必需的数字逻辑和必要的模拟电路。它可以实现全速 USB 主机或者外设传输，并兼容 USB 规范 2.0 版本。 MAX3421E 通过标准串行外设接口（SPI）的四根线与任何微控制器相连接，然后使用一组寄存器进行操作。 精确时间的 USB 的操作都在 MAX3421E 内部完成，使得 SPI 主体并不需要处理精确的 USB 时序要求。

9.10.1 操作原理 – SPI

为了与 MAX3421E 工作，我们需要使用 SPI 连接来对微控制器进行对接接口。 SPI 是微控制器常用的协议，用于在短距离上快速地与一个或多个外围设备进行通信。这是一个主机与从机通信的同步串行协议。主机对所有设备用三条常规的线，并对每一个设备使用一根特殊的线连接。这四个连接线分别是：

- MOSI (Master Out Slave In) – 向从机传输数据的主线
- MISO (Master In Slave Out) – 向主机传输数据的从线
- SCLK (Serial Clock) – 主机用于同步数据传输的同步信号
- SS (Slave Select) – 低电平有效的使能线，主机从而可以控制从机的开闭

该 SPI 标准是不严格的，每个不同的设备对它的实现略有不同。基本上有四种模式的传输，它们控制数据是在上升沿还是下降沿（称为时钟相位）变化，以及时钟空闲时是高电平或低电平（称为时钟极性）。MAX3421E 的使用模式 0，意味着 MOSI 和 MISO 线上的数据在下降 SCK 沿变化，并且 SCK 时钟空闲时为低电平。此外，MAX3421E 在串行线上首先发送最高显著位（MSB），并可以在速度高达 26MHz 时工作。

SPI 主机通过读取和写入芯片内部寄存器访问 MAX3421E。一个典型的数据传输包括写入第一个命令字节，用于设置寄存器地址和数据方向，然后传输附加字节读取或写入数据到寄存器或内部 FIFO 中。第一个命令字节的格式如图 9.49。前五位用于确定要访问的寄存器的地址， R'/ W 位确定到底是读取还是写入数据—0 为读，1 为写。

Reg4	Reg3	Reg2	Reg1	Reg0	0	R'/W	0

图 2.1 第一个命令字节的格式

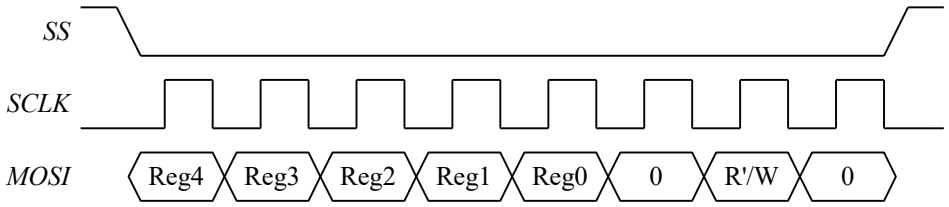


图 2.2 通过 SPI 总线传输一个字节的时序图。

9.10.2 操作原理 – USB

USB 协议比 RS-232 协议复杂得多。然而，低级详细的实现 USB 协议由 MAX3421E 的内部组装串行接口引擎（SIE）执行。 这包括

- USB 包 PID 侦查和检错
- CRC 检错和生成
- 在错误的情况下自动重试
- USB 包生成
- NRZI 数据编码与译码
- 位填充和拆箱
- USB 错误检测
- USB 总线复位，挂起和唤醒检测
- USB 暂停/恢复信号
- 自动流量控制(NAK).

该 MAX3421E 还具有一个内部振荡器和锁相环（PLL），它可以处理 USB 的时间严格的操作。为了使用 MAX3421E 实现一个 USB 主机控制器，我们只需遵循和执行常规的 USB 主机操作，通过设置外围设备地址，端点，请求类型，请求和发送包等操作。这是通过设置 MAX3421E 的内部相应的寄存器来完成的。